# Project - High Level Design

# On

# Multi-Agent Education system

Course Name: Agentic AI

***Institution Name:*** Medicaps University – Datagami Skill Based Course

*Student Name(s) & Enrolment Number(s):*

| Sr no | Student Name | Enrolment Number |
|---|---|---|
| 1. | SARTHAK PATEL | EN22CS301877 |
| 2. | SAKSHI JAIN | EN22CS301845 |
| 3. | SHREYA MAHODAY | EN22CS301930 |
| 4. | SHREYA THAKKAR | EN22CS301933 |
| 5. | SIYA KUSUMAKAR | EN22CS301960 |

*Group Name: 09D8*

*Project Number: AAI-44*

*Industry Mentor Name: Aashruti Shah*

*University Mentor Name: Prof Pradeep Baniya*

*Academic Year: 2025-2026*

# Table of Contents


1. Introduction.
   - 1.1. Scope of the document.
   - 1.2. Intended Audience
   - 1.3. System overview.
2. System Design.
   - 2.1. Application Design
   - 2.2. Process Flow.
   - 2.3. Information Flow.
   - 2.4. Components Design
   - 2.5. Key Design Considerations
   - 2.6. API Catalogue.
3. Data Design.
   - 3.1. Data Model
   - 3.2. Data Access Mechanism
   - 3.3. Data Retention Policies
   - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
   - 7.1. Security Aspects
   - 7.2. Performance Aspects
8. References

# 1. Introduction

The Multi-Agent Education System is an intelligent, AI-driven platform designed to automatically generate structured academic study notes using a collaborative agent-based architecture. The system leverages multiple specialized AI agents that work together to research, analyze, and organize educational content based on a given topic.

Traditional AI-based content generation systems often rely on a single large language model call, which may produce unstructured or inconsistent outputs. This project addresses those limitations by implementing a multi-agent framework where each agent performs a specific role and communicates with other agents to complete the overall educational content generation workflow.

The system is designed to improve content clarity, logical structuring, modularity, and scalability. By dividing responsibilities between agents such as the Researcher Agent and the Writer Agent, the system ensures higher quality outputs and better task orchestration.

The platform demonstrates advanced inter-agent communication, structured hand-off protocols, sequential workflow execution, and integration with Large Language Models (LLMs) through an API gateway.

## 1.1 Scope of the Document

This document provides a High-Level Design (HLD) overview of the Multi-Agent Education System. It describes the overall architecture, system components, agent interactions, and major functional capabilities.

The scope of this document includes:

- Overview of the system architecture
- Description of individual agents and their responsibilities
- High-level workflow of study note generation
- Interaction between agents and external LLM services
- Key design decisions and technology stack
- Assumptions and system boundaries

## 1.2 Intended Audience

This document is intended for the following stakeholders:

1. Developers
   To understand the architecture, agent roles, and interaction flow before implementing or modifying system components.
2. Project Reviewers / Evaluators
   To evaluate the technical design, scalability, modularity, and innovation of the multi-agent approach.
3. System Architects
   To review architectural decisions, agent orchestration mechanisms, and integration patterns.
4. Academic Mentors
   To understand how Agentic AI concepts are applied in the education domain.
5. QA / Testing Team
   To identify system modules and define test scenarios based on functionality.

## 1.3 System Overview

The Multi-Agent Education System is built on a collaborative multi-agent architecture where each agent is responsible for a specific task in the educational content generation process.

**Core Concept**

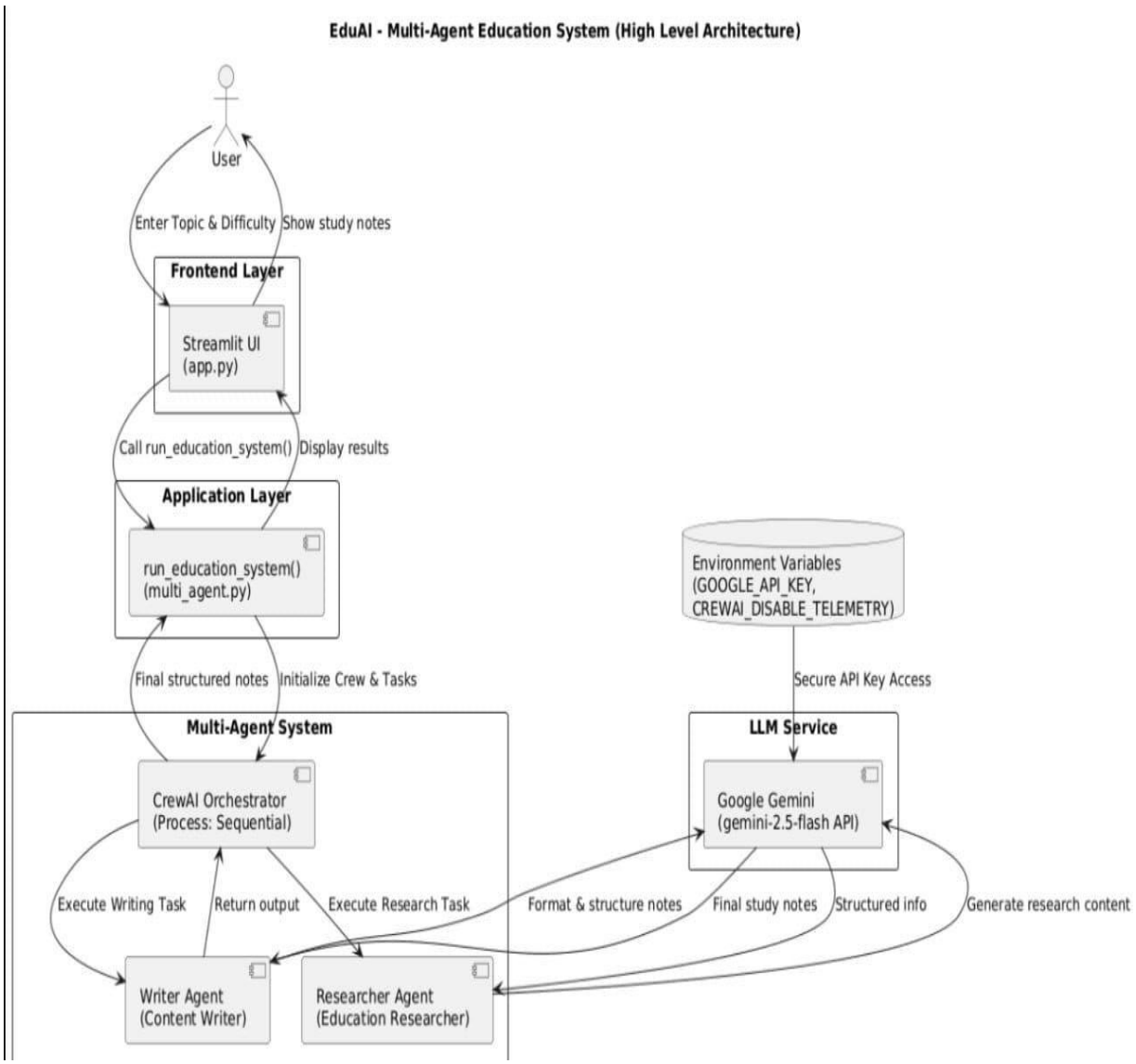
Instead of using a monolithic AI system, the platform is divided into specialized agents such as:

• Researcher Agent – Collects detailed academic information related to the topic.
• Writer Agent – Synthesizes and formats research into structured study notes.
• Orchestrator (CrewAI) – Manages communication and task delegation among agents.

Each agent operates independently but communicates through structured task delegation mechanisms. This modular structure improves scalability, maintainability, and extensibility of the system.

**High-Level Architecture Flow**

1. The user enters a topic through an interface (CLI/Web UI).
2. The CrewAI Orchestrator receives the request.
3. The request is delegated to the Researcher Agent.
4. The Writer Agent processes the research output.
5. The final structured study notes are generated and returned to the user.



# 2. System Design

The system is designed using a modular, sequential multi-agent architecture to ensure clarity and scalability.

## 2.1 Application Design

The architecture consists of four layers:

**Presentation Layer**

Handles user interaction (CLI/Streamlit/Web UI).

**Application Layer**

Implements CrewAI orchestration logic.

**Agent Layer**

Contains Researcher and Writer agents.

**LLM Integration Layer**

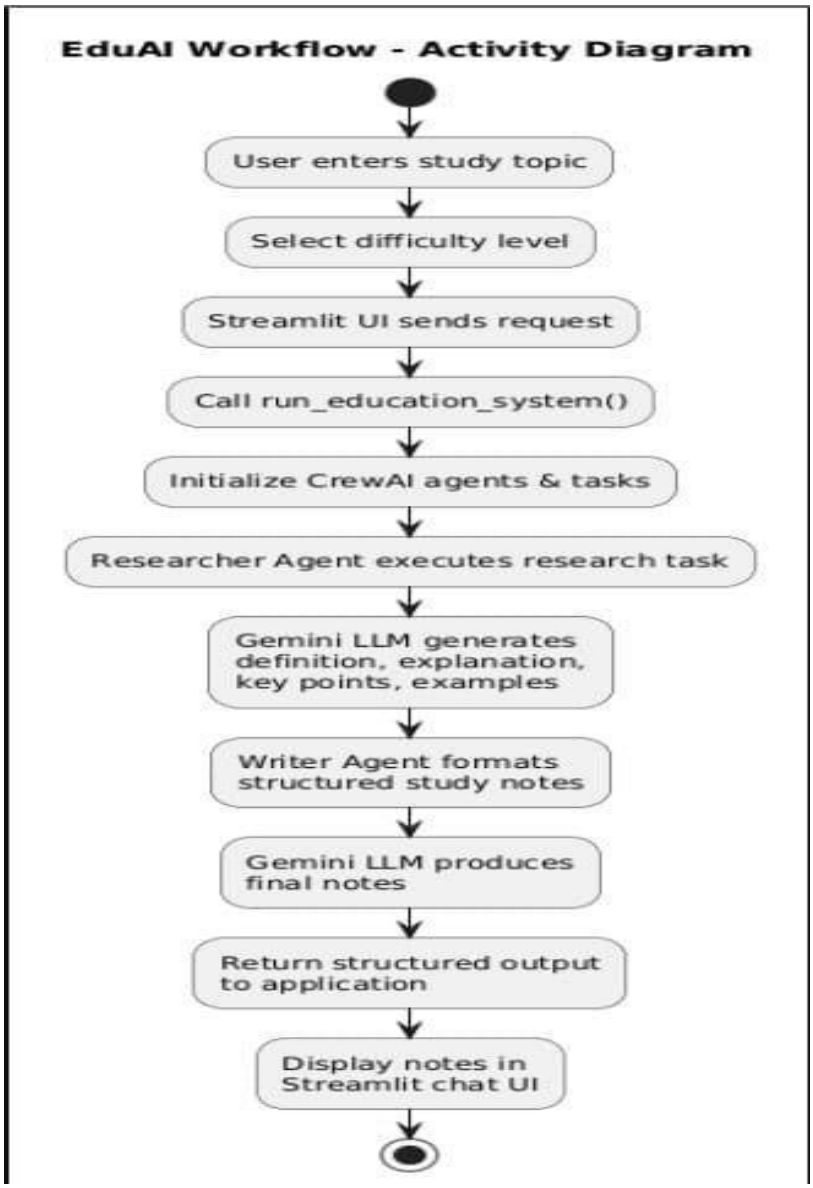
Uses OpenRouter API to access Google Gemini 2.0 Flash model.

This layered design ensures separation of concerns and maintainability.

## 2.2 Process Flow

1. User submits topic.
2. Orchestrator initializes agents.
3. Researcher Agent generates academic research.
4. Writer Agent organizes content into structured format.
5. Output returned to user.

Sequential execution ensures logical dependency handling.



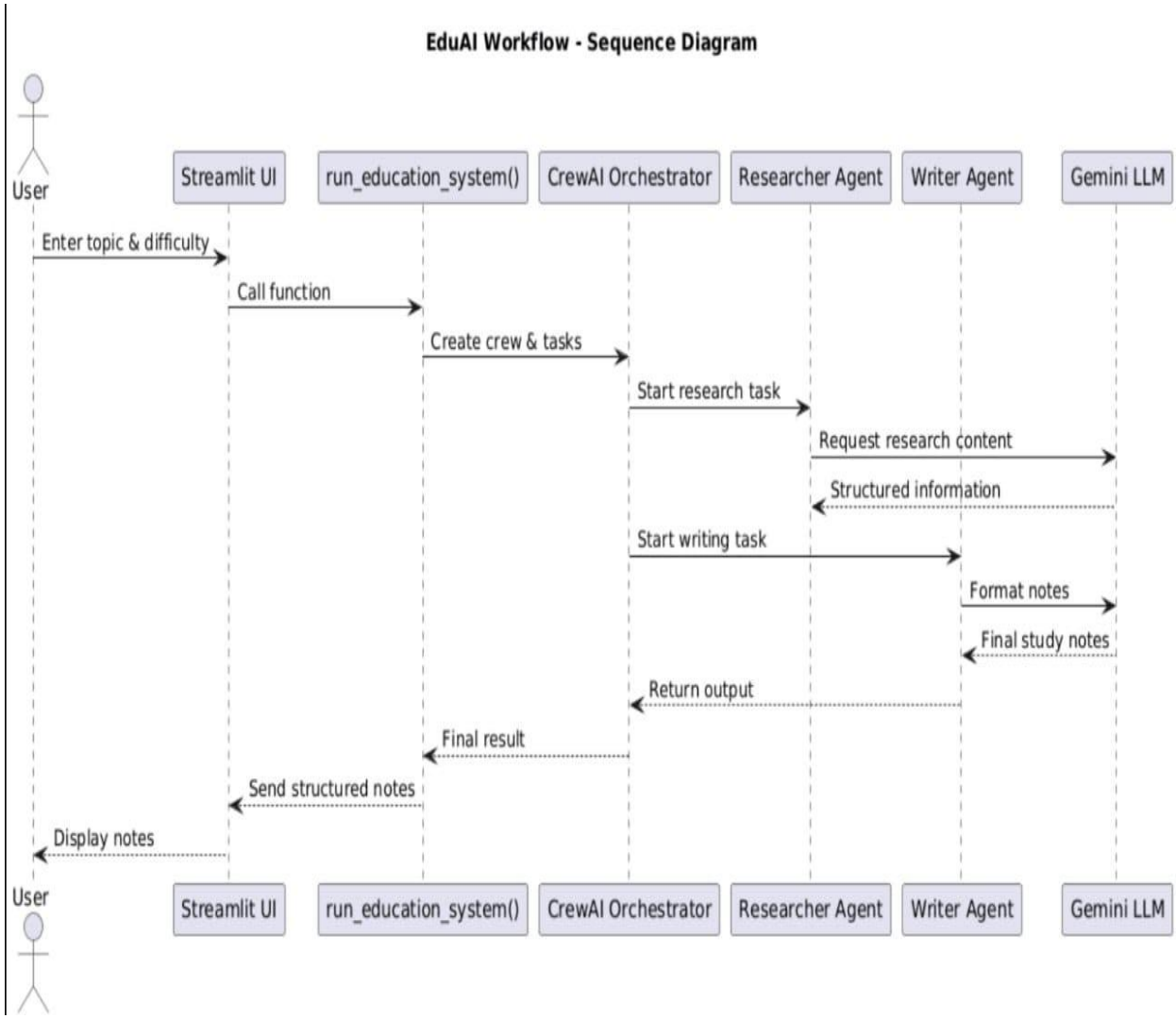
## 2.3 Information Flow

Input: Topic (String)

Processing:
• Task creation
• Agent reasoning
• Sequential execution
• LLM prompt handling

Output: Structured Study Notes

Information flows:
User → Orchestrator → Researcher → Writer → LLM → Output



## 2.4 Components Design

**Orchestrator Component**
Coordinates agent execution and task flow.

**Researcher Agent**
Performs detailed content analysis and explanation.

**Writer Agent**
Formats structured notes including:
• Definition
• Explanation
• Key Points
• Examples
• Summary

**LLM Gateway (OpenRouter)**
Handles API communication.

**Environment Configuration Module**
Manages secure API keys.

Each component is loosely coupled to ensure flexibility.

## 2.5 Key Design Considerations

• Modular multi-agent architecture
• Clear role separation
• Sequential workflow for accuracy
• Secure environment-based configuration
• Easy scalability for adding more agents
• LLM abstraction through API gateway

## 2.6 API Catalogue

| Category | API | Method | Description |
|---|---|---|---|
| Content Generation | Generate Notes | POST | Accepts topic and returns structured notes |

External Integration:
• OpenRouter API
• Google Gemini 2.0 Flash Model

All APIs use JSON-based request and response format.

# 3. Data Design

The system primarily uses in-memory data structures.

## 3.1 Data Model

Entities include:

• Topic
• Research Content
• Structured Notes
• Task Object
• Execution Metadata

No persistent database is currently implemented.

## 3.2 Data Access Mechanism

• JSON-based input/output
• Internal Python objects
• Secure API headers
• Environment-based key storage

## 3.3 Data Retention Policies

• No long-term data storage
• Runtime-only processing
• No personal data collection
• Session-based execution

## 3.4 Data Migration

Not applicable in current academic scope.

Future scope:
• Database integration
• Cloud deployment
• Data versioning

# 4. Interfaces

## User Interface

• Topic Input Field
• Generate Button
• Structured Output Display

## System Interface

• REST-based communication
• JSON request/response

## Internal Agent Communication

Agents communicate through orchestrator using structured task messages.

# 5. State and Session Management

• The system follows a **stateless architecture**, where each request is processed independently without storing long-term session data.

• When a user submits a topic, the orchestrator initializes the required agents, executes the workflow, and returns the response within the same execution cycle.

• No persistent user session or server-side state is maintained after the response is generated.

• Clearly defined workflow states are followed to maintain structured execution:

- REQUEST_INITIATED
- RESEARCH_COMPLETED
- WRITING_COMPLETED
- RESPONSE_GENERATED
- COMPLETED

• Each state transition occurs only after successful completion of the previous step, ensuring logical consistency in output generation.

• This stateless design improves scalability, reliability, and ease of deployment.

# 6. Caching

• Currently, the system does not implement caching, as it is designed as an academic prototype focusing on agent collaboration.

• Every new topic triggers fresh execution of the Researcher and Writer agents.

• In future enhancements, topic-based caching can be implemented to store previously generated study notes for repeated queries.

• Intermediate agent outputs may also be cached within a request cycle to reduce redundant processing.

• Caching can help reduce LLM API calls, improve response time, and optimize overall system performance.

• Tools like Redis or in-memory caching mechanisms can be integrated in future versions.

# 7. Non-Functional Requirements

## 7.1 Security Aspects

• API keys are securely stored in environment variables (.env file) and are not hardcoded in the source code.

• All communication with external APIs such as OpenRouter is performed over secure HTTPS connections.

• User inputs are validated before processing to prevent invalid or harmful requests.

• Sensitive configurations are restricted to authorized system components only.

• The system avoids storing personal user data, reducing privacy risks.

### 7.2 Performance Aspects

• The system uses a lightweight Python backend for efficient processing.

• Sequential agent execution ensures structured and predictable output generation.

• Average response time ranges between 3–6 seconds depending on topic complexity.

• The stateless design reduces memory usage and allows easy horizontal scalability.

• Efficient agent hand-off minimizes processing delay between research and writing stages.

• The system can handle multiple independent requests in moderate workloads.

## 8. References

1. Medicaps University – Datagami Skill Based Course, Agentic AI Project GuidelinesCrewAI Documentation
2. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson Education.
3. Wooldridge, M. (2009). *An Introduction to MultiAgent Systems*. John Wiley & Sons.
4. CrewAI Documentation. (2024). Multi-Agent Orchestration Framework Guide.
5. OpenRouter Documentation. (2024). API Reference and Model Integration Guide.
6. Google AI. (2024). Gemini 2.0 Flash Model – Technical Documentation.
7. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral Dissertation, University of California, Irvine).